2.2.11 Defining Functions that Remember Values They Have Found

When you make a function definition using `:=`, the value of the function is recomputed every time you ask for it. In some kinds of calculations, you may end up asking for the same function value many times. You can save time in these cases by having *Mathematica* remember all the function values it finds. Here is an "idiom" for defining a function that does this.

`f[x_] := f[x] =` *rhs*	define a function which remembers values that it finds

Defining a function that remembers values it finds.

This defines a function `f` which stores all values that it finds.

```
In[1]:= f[x_] := f[x] = f[x - 1] + f[x - 2]
```

Here are the end conditions for the recursive function `f`.

```
In[2]:= f[0] = f[1] = 1
Out[2]= 1
```

Here is the original definition of `f`.

```
In[3]:= ?f

f
f/: f[1] = 1
f/: f[0] = 1
f/: f[x_] := f[x] = f[x - 1] + f[x - 2]
```

This computes `f[5]`. The computation involves finding the sequence of values `f[5]`, `f[4]`, ... `f[2]`.

```
In[3]:= f[5]
Out[3]= 8
```

All the values of `f` found so far are explicitly stored.

```
In[4]:= ?f

f
f/: f[1] = 1
f/: f[0] = 1
f/: f[2] = 2
f/: f[3] = 3
f/: f[4] = 5
f/: f[5] = 8
f/: f[x_] := f[x] = f[x - 1] + f[x - 2]
```

If you ask for `f[5]` again, *Mathematica* can just look up the value immediately; it does not have to recompute it.

```
In[4]:= f[5]
Out[4]= 8
```

You can see how a definition like `f[x_] := f[x] = f[x-1] + f[x-2]` works. The function `f[x_]` is defined to be the "program" `f[x] = f[x-1] + f[x-2]`. When you ask for a value of the function `f`, the "program" is executed. The program first calculates the value of `f[x-1] + f[x-2]`, then saves the result as `f[x]`.

It is often a good idea to use functions that remember values when you implement mathematical *recursion relations* in *Mathematica*. In a typical case, a recursion relation gives the value of a function f with an integer argument x in terms of values of the same function with arguments $x-1$, $x-2$, etc. The Fibonacci function definition $f(x) = f(x-1) + f(x-2)$ used above is an example of this kind of recursion relation. The point is that if you calculate say $f(10)$ by just applying the recursion relation over and over again, you end up having to recalculate quantities like $f(5)$ many times. In a case like this, it is therefore better just to *remember* the value of $f(5)$, and look it up when you need it, rather than having to recalculate it.

There is of course a trade-off involved in remembering values. It is faster to find a particular value, but it takes more memory space to store all of them. You should usually define functions to remember values only if the total number of different values that will be produced is comparatively small.

Defining functions that remember values they have found is sometimes called "dynamic programming".